

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5– Naturalization (BL5,BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	CH.V.Ramayogi	Reg. No.:	192424319
Section / Batch:	Slot-B-AIDS/2024-28	Date:	26-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

Code of Conduct

I CH.V.Ramayogi/192424319(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: CH.V.Ramayogi

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%				
Constraint Analysis	25%				
Solution Development	25%				
Application of Concepts	15%				
Justification	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q7. Sudoku Solver Using Backtracking
CO5-AT1: Constraint-Based Problem Solving – Slot-B
CH.V.Ramayogi-192424319

1. Introduction

Sudoku is a constraint-based problem in which a partially filled 9×9 grid must be completed by placing numbers from 1 to 9 in the empty cells. The final solution must satisfy three major constraints: every row must contain numbers 1–9 without repetition, every column must contain numbers 1–9 without repetition, and every 3×3 subgrid must contain numbers 1–9 without repetition.

Sudoku is a well-known NP-complete problem in its generalized form. A backtracking algorithm can efficiently solve many standard 9×9 Sudoku instances by systematically trying possible values and rejecting invalid choices early.

2. Input Conditions

The input is a 9×9 Sudoku grid.

- A value from 1 to 9 represents a pre-filled cell.
- 0 represents an empty cell.
- The initial grid must not violate Sudoku constraints.
- There must be no duplicate non-zero values in any row, column, or 3×3 subgrid.

Example Input

```
5 3 0 0 7 0 0 0 0
6 0 0 1 9 5 0 0 0
0 9 8 0 0 0 0 6 0
8 0 0 0 6 0 0 0 3
4 0 0 8 0 3 0 0 1
7 0 0 0 2 0 0 0 6
0 6 0 0 0 0 2 8 0
0 0 0 4 1 9 0 0 5
0 0 0 0 8 0 0 7 9
```

3. Constraints

For an empty cell (r,c) , a number x can be placed only if:

Row Constraint: x must not already exist in row r .

Column Constraint: x must not already exist in column c

Subgrid Constraint: x must not already exist in the corresponding 3×3 subgrid.

Therefore:

$\text{Possible}(r,c) = \{1,2,3,4,5,6,7,8,9\} - \text{Row}(r) - \text{Column}(c) - \text{Subgrid}(r,c)$

The intersection of these constraints significantly reduces the number of possible values for each empty cell.

4. Interaction of Constraints

The three constraints work together. For example, suppose an empty cell has the following restrictions:

- Row already contains {1, 3, 5, 7}
- Column already contains {2, 4, 8}
- Its 3×3 subgrid contains {1, 2, 6}

After applying all constraints, the only possible value is {9}. Therefore, the cell must contain 9. This is called constraint propagation because assigning a value to one cell reduces the possible values of other cells.

5. Backtracking Approach

Backtracking constructs the solution incrementally.

1. Find an empty cell.
2. Determine its possible values.
3. Try one possible value.
4. Check whether the value satisfies all constraints.
5. If valid, place the value.
6. Recursively solve the remaining grid.
7. If no solution is possible, remove the value.
8. Try the next possible value.
9. Continue until the entire grid is filled.

Basic Algorithm

SudokuSolver(grid):

 Find an empty cell

 If no empty cell exists:

 return TRUE

 Select the empty cell

 For number = 1 to 9:

 If number is valid:

 Place number

 If SudokuSolver(grid) is TRUE:

```
    return TRUE
```

```
    Remove number
```

```
    return FALSE
```

6. C++ Implementation

```
#include <iostream>
```

```
using namespace std;
```

```
bool isSafe(int grid[9][9], int row, int col, int num)
```

```
{
```

```
    // Check row
```

```
    for (int x = 0; x < 9; x++)
```

```
    {
```

```
        if (grid[row][x] == num)
```

```
            return false;
```

```
    }
```

```
    // Check column
```

```
    for (int x = 0; x < 9; x++)
```

```
    {
```

```
        if (grid[x][col] == num)
```

```
            return false;
```

```
    }
```

```
    // Check 3x3 subgrid
```

```
    int startRow = row - row % 3;
```

```
    int startCol = col - col % 3;
```

```
    for (int i = 0; i < 3; i++)
```

```
    {
```

```
        for (int j = 0; j < 3; j++)
```

```
        {
```

```
            if (grid[startRow + i][startCol + j] == num)
```

```
                return false;
```

```
        }
```

```
    }
```

```
    return true;
```

```
}
```

```

bool solveSudoku(int grid[9][9])
{
    int row = -1;
    int col = -1;
    bool emptyCell = false;

    // Find an empty cell
    for (int i = 0; i < 9; i++)
    {
        for (int j = 0; j < 9; j++)
        {
            if (grid[i][j] == 0)
            {
                row = i;
                col = j;
                emptyCell = true;
                break;
            }
        }
        if (emptyCell)
            break;
    }
    // No empty cell means solution is complete
    if (!emptyCell)
        return true;
    // Try numbers 1 to 9
    for (int num = 1; num <= 9; num++)
    {
        if (isSafe(grid, row, col, num))
        {
            grid[row][col] = num;
            if (solveSudoku(grid))
                return true;
            // Backtrack

```

```

        grid[row][col] = 0;
    }
}
return false;
}
void printGrid(int grid[9][9])
{
    for (int i = 0; i < 9; i++)
    {
        for (int j = 0; j < 9; j++)
        {
            cout << grid[i][j] << " ";
        }
        cout << endl;
    }
}
int main()
{
    int grid[9][9] =
    {
        {5,3,0,0,7,0,0,0,0},
        {6,0,0,1,9,5,0,0,0},
        {0,9,8,0,0,0,0,6,0},
        {8,0,0,0,6,0,0,0,3},
        {4,0,0,8,0,3,0,0,1},
        {7,0,0,0,2,0,0,0,6},
        {0,6,0,0,0,0,2,8,0},
        {0,0,0,4,1,9,0,0,5},
        {0,0,0,0,8,0,0,7,9}
    };
    if (solveSudoku(grid))
    {
        cout << "Sudoku Solution:" << endl;
        printGrid(grid);
    }
}

```

```

    }
    else
    {
        cout << "No solution exists." << endl;
    }
    return 0;
}

```

7. Constraint Propagation and Pruning

Constraint propagation is used to reduce the search space before and during backtracking.

Minimum Remaining Values (MRV): Instead of selecting the first empty cell, select the empty cell having the fewest possible values. For example, if Cell A has {1,3,7}, Cell B has {4}, and Cell C has {2,5}, Cell B should be selected first because it has only one possible value.

Forward Checking: After assigning a value to a cell, remove that value from the candidate sets of related cells. If any neighboring cell has zero possible values, the current assignment is immediately rejected.

Early Pruning: A branch is abandoned as soon as a row contains a duplicate, a column contains a duplicate, a subgrid contains a duplicate, or an empty cell has no valid candidate. This prevents unnecessary exploration of invalid branches.

8. Correctness Justification

The algorithm is correct because every value inserted into an empty cell is checked against the corresponding row, column, and 3×3 subgrid.

If the algorithm reaches a completely filled grid, every cell contains a value from 1–9 and all three Sudoku constraints have been satisfied. Hence, the resulting grid is a valid Sudoku solution.

If a selected value eventually leads to an invalid configuration, the algorithm removes that value and tries another possibility. Therefore, valid solutions are not permanently discarded.

If no candidate works for a cell, the algorithm backtracks to a previous decision. If all possibilities are exhausted, the algorithm correctly concludes that no solution exists.

9. Performance Analysis

Let n represent the number of empty cells. In the worst case, each empty cell can have up to 9 possible values. Therefore, the worst-case search space is approximately $O(9^n)$.

For a standard 9×9 Sudoku, there are at most 81 cells, so the theoretical worst case can be

very large. However, practical performance is much better because row, column, and subgrid constraints remove invalid values, while constraint propagation, MRV, and backtracking prune invalid branches.

The space complexity is mainly due to the recursion stack and the Sudoku grid. The recursion depth is $O(n)$, with additional storage depending on the candidate-tracking implementation.

10. Performance Challenges

The main challenge is the potentially exponential search space. A Sudoku with many empty cells may require exploring a large number of combinations.

Performance can be improved using:

- MRV heuristic
- Forward checking
- Constraint propagation
- Bit masks for fast row, column, and subgrid checks
- Degree heuristic for choosing highly constrained cells

These techniques reduce the number of recursive calls and make the solver considerably faster.

11. Conclusion

The Sudoku problem demonstrates how constraint-based problem solving can be combined with backtracking to solve a difficult combinatorial problem. Each empty cell has a domain of possible values, while row, column, and subgrid rules act as constraints. Constraint propagation and pruning reduce the search space, while backtracking guarantees that alternative choices are explored when necessary. Although the worst-case complexity is exponential, effective heuristics such as MRV and forward checking make the approach practical for most 9×9 Sudoku puzzles.